SENSEX MFT SYSTEM

Master Architecture Blueprint v3.0 — Upstox Edition

Data Ingestion · Pre-processing · Feature Engineering · Normalization · Memory-Optimized · 16 GB RAM

API: Upstox v2 SDK (Python)
RAM Budget: 16 GB (Optimized Pipeline)
Math Standard: Institutional-Grade Quant
Version: 3.0 | March 2026

SECTION 1 — Upstox API Integration & Access Token Protocol

The pipeline uses the official Upstox v2 Python SDK exclusively. The Upstox access token is OAuth2-based, expires daily, and must NEVER be hardcoded or stored in version control.

1.1 SDK Installation & Dependencies

INSTALL

pip install upstox-python-sdk
pip install upstox-python-sdk --upgrade # ensure >= 2.x

Core imports:

```
import upstox_client
from upstox_client.rest import ApiException
configuration = upstox_client.Configuration()
configuration.access_token = <runtime_token>
```

1.2 Access Token — Runtime Prompt Protocol

The code builder MUST implement the following token acquisition flow. The access token is valid for ONE trading day (resets at midnight IST). It must be obtained interactively at the start of each run.

TOKEN FLOW

Step 1 — FIRST RUN (One-time API key setup):

```
Store UPSTOX_API_KEY and UPSTOX_API_SECRET in .env file only.
Never store access_token in .env (it changes daily).
```

Step 2 — DAILY RUN (Runtime prompt):

```
The program starts and checks for token in session cache.
If absent or expired: prompt user interactively.
Code: access_token = getpass.getpass('Enter Upstox Access Token: ')
Validate: call upstox_client.UserApi.get_profile() to confirm token.
Cache in memory (os.environ['UPSTOX_TOKEN']) for the session only.
NEVER write access_token to disk, logs, or any file.
```

Step 3 — Token Expiry Handling:

```
Wrap all API calls in try/except ApiException.
```

```
On HTTP 401: clear cache, re-prompt for new token, retry once.
On second 401: ABORT and log error — do not loop infinitely.
```

1.3 Upstox API Endpoints Used

Data Type	Upstox SDK Class	Method	Notes
1-min OHLCV (Index)	HistoryApi	get_intra_day_candle_data()	instrument_key='NSE_INDEX|Nifty 50' or 'BSE_INDEX|SENSEX'
1-min OHLCV (Stocks)	HistoryApi	get_intra_day_candle_data()	instrument_key='BSE_EQ|RELIANCE' etc. Rate: 10 req/s
Historical OHLCV	HistoryApi	get_historical_candle_data()	interval='1minute', for backfill Jan 2024 onward
Option Chain	OptionChainApi	get_option_chain()	expiry_date param; returns all strikes with IV+Greeks
Market Quote (Live)	MarketQuoteApi	get_market_quote_ohlc()	For live streaming during inference
User Profile (Auth check)	UserApi	get_profile()	Used to validate access token on startup

1.4 Upstox Instrument Key Reference (Sensex Constituents)

Instrument keys follow the pattern: EXCHANGE_SEGMENT|TRADING_SYMBOL. The code builder must generate all 30 keys from a master lookup CSV at runtime, not hardcode them.

KEY FORMAT

Index: BSE_INDEX|SENSEX
Equity: BSE_EQ|{SYMBOL} (e.g., BSE_EQ|RELIANCE, BSE_EQ|HDFCBANK)
Option Call: BSE_FO|SENSEX{YYMMDD}C{STRIKE} (example: BSE_FO|SENSEX250130C80000)
Option Put: BSE_FO|SENSEX{YYMMDD}P{STRIKE}

IMPORTANT: Upstox instrument keys are available via:

```
upstox_client.InstrumentsApi.get_instruments(exchange='BSE')
Save the full instrument master to ./data/instruments/bse_instruments.csv
Refresh this file at start of each new month.
```

SECTION 2 — 16 GB RAM Architecture: Zero-Bottleneck Design

A 16 GB RAM system can run the full pipeline without swapping IF data is processed in partitioned chunks and intermediate objects are explicitly released. The design below ensures peak RAM usage stays under 10 GB, leaving 6 GB for OS + Python interpreter overhead.

2.1 Memory Budget Per Stage

Pipeline Stage	Est. Data Size	Strategy	Peak RAM Usage
Raw Ingestion (1 day)	~45 MB (30 stocks × 375 bars × float32)	Process 1 day at a time, persist immediately	< 500 MB
Pre-processing (batch)	~1.2 GB (1 month)	Process week-by-week, use float32 everywhere	< 2.5 GB
Feature Engineering	~2 GB (1 month with options)	Chunked groupby, release IV arrays after spline fit	< 4 GB

Normalization (all splits)	~3 GB (2 years)	Load one split at a time, fit → transform → save → del	< 6 GB
Full 2-year dataset (flat)	~8 GB uncompressed	NEVER load full dataset at once — always chunk	N/A — forbidden

2.2 Core Memory-Optimization Techniques

Technique 1: Enforce float32 Everywhere

CODE PATTERN

```
# On ingestion — convert immediately after API response:
df = df.astype({
    'open': 'float32', 'high': 'float32',
    'low': 'float32', 'close': 'float32',
    'volume': 'int32'   # volume never needs float
})
# int32 volume saves 50% vs int64 (pandas default)
# float32 OHLC saves 50% vs float64 (pandas default)
# A 2-year, 30-stock dataset: float64=12GB vs float32=6GB
```

Technique 2: Parquet Column Pruning

- Never load the full master table when only a subset of columns is needed.
- Always pass the columns= parameter to pd.read_parquet().
- Example: Normalization engine reads only feature columns, not raw OHLC.

CODE PATTERN

```
# WRONG — loads everything (~2GB for 1 month):
df = pd.read_parquet('features_jan.parquet')

# CORRECT — loads only needed columns (~400MB):
LNN_COLS = ['timestamp','idx_log_ret','synth_atm_iv','wrs_residual',
            'sector_bank_flow','idx_rv5m','idx_target_1m']
df = pd.read_parquet('features_jan.parquet', columns=LNN_COLS)
```

Technique 3: Explicit Garbage Collection After Each Stage

CODE PATTERN

```
import gc

def process_week(week_files):
    raw_df = load_and_merge(week_files)     # ~500 MB
    clean_df = preprocess(raw_df)
    del raw_df; gc.collect()                # Free 500 MB immediately
    feat_df = compute_features(clean_df)
    del clean_df; gc.collect()              # Free processed intermediate
    save_parquet(feat_df)
    del feat_df; gc.collect()               # Free before next week
```

Technique 4: Chunked Daily Processing (Primary Loop)

- The MASTER loop in pipeline.py iterates ONE TRADING DAY at a time.
- Each day: ingest → preprocess → feature-compute → persist → clear memory.
- Never accumulate multiple days of data in RAM before persisting.
- Use pathlib.Path.glob() to find files dynamically, not os.listdir().

Technique 5: Chunked Parquet Reading for Multi-Month Operations

CODE PATTERN

```
import pyarrow.parquet as pq

# Stream parquet in row-groups instead of loading full file:
pf = pq.ParquetFile('features_2024.parquet')
for batch in pf.iter_batches(batch_size=50_000):  # ~50k rows = ~200 MB
    df_chunk = batch.to_pandas()
    df_chunk = df_chunk.astype({'col': 'float32'})
    process_and_append_to_result(df_chunk)
    del df_chunk; gc.collect()
```

Technique 6: Rolling Scaler State (Avoid Full-Fit Memory)

- For the rolling Z-score, never store the full 60-bar history per feature. Only store the rolling mean and std as two float32 values per feature per session.
- Use Welford's online algorithm for numerically stable streaming mean/variance without storing the full window.

WELFORD ONLINE ALGORITHM

```
class WelfordRolling:
    # Computes rolling mean/variance in O(1) memory per feature
    # Numerically stable (no catastrophic cancellation)
    def __init__(self, window): self.n=0; self.mean=0; self.M2=0;
self.window=window
    def update(self, x_new, x_old):  # x_old = value leaving the window
        self.n = min(self.n+1, self.window)
        delta = x_new - x_old
        self.mean += delta / self.n
        self.M2 += (x_new - x_old) * (x_new - self.mean + x_old - (self.mean -
delta/self.n))
    @property
    def std(self): return (self.M2 / max(self.n-1,1)) ** 0.5
```

Technique 7: Option Chain Memory Control

- The option chain is the largest data structure (~150 columns × 375 bars × 20 strikes).
- Do NOT store raw option OHLCV in the master table. Only store DERIVED features: synth_atm_iv, iv_skew, greeks, PCR.
- Process each 1-minute bar's option chain, extract features, then discard raw option rows.
- This reduces option-related memory from ~3 GB to ~200 MB per month.

SECTION 3 — Advanced Mathematical Logic & Formulas

All formulas are specified to institutional-grade precision. Each formula includes: definition, implementation note, numerical stability consideration, and edge case handling.

3.1 Log Returns — Multi-Scale with Overnight Separation

LOG RETURN FRAMEWORK

```
# Intraday 1-min return (NaN at session open):
r_t = ln(P_t / P_{t-1})        [undefined at bar 0 of each session]

# Overnight gap return (computed separately):
r_overnight = ln(P_09:15_today / P_15:29_yesterday)
```

```
# Multi-scale momentum (rolling sums of r_t, within session):
M_5  = sum(r_{t-4} ... r_t)    [5-bar = 5-min momentum]
M_15 = sum(r_{t-14} ... r_t)   [15-bar momentum]
M_30 = sum(r_{t-29} ... r_t)   [30-bar momentum]

# Momentum Decay Signal:
MDS_t = (M_5 - M_15) / max(|M_15|, epsilon)   [normalized decay]
# MDS < -0.5: strong exhaustion. MDS > 0.5: acceleration.

# Numerical stability: add epsilon=1e-10 before ln to prevent ln(0)
r_t = ln((P_t + 1e-10) / (P_{t-1} + 1e-10))
```

3.2 Session-Aware Rolling Z-Score (Corrected & Hardened)

ROLLING Z-SCORE — PRODUCTION FORMULA

```
# Standard formula:
z_t = (x_t - mu_rolling(t, W)) / max(sigma_rolling(t, W), epsilon)

# Where:
  mu_rolling(t, W)    = mean(x_{t-W+1} ... x_t) — SESSION ONLY
  sigma_rolling(t, W) = std(x_{t-W+1} ... x_t)  — SESSION ONLY
  W = 60 (bars), epsilon = 1e-8 (prevents division by zero)

# Session boundary enforcement:
  At bar index k < W within session: use EXPANDING window [0..k]
  NEVER use bars from previous session in rolling window
  Implementation: df.groupby('session_date').transform(
      lambda s: (s - s.expanding(min_periods=5).mean()) /
                 s.expanding(min_periods=5).std().clip(lower=1e-8))

# Clip AFTER z-scoring:
  z_t_clipped = clip(z_t, -3.5, 3.5)
  [3.5-sigma not 3.0 — 3.0 clips too aggressively on BSE open volatility]

# Store rolling state for live inference (no warm-up lag):
  roll_mu_60[t]  = mu_rolling(t, 60)  — save as float32 column
  roll_std_60[t] = sigma_rolling(t, 60) — save as float32 column
```

3.3 Synthetic IV Surface — PCHIP with Arbitrage-Free Constraints

IV SURFACE CONSTRUCTION

```
# Step 1: Compute implied volatility per strike using Newton-Raphson
# (py_vollib uses Brent's method — numerically robust):
  sigma_i = implied_vol(C_market, S, K_i, T, r, flag)
  T = max(DTE_calendar / 365.0, 5/(375*365))  [min 5-min to expiry]

# Step 2: Map to standardized moneyness (log-forward moneyness):
  F = S * exp(r * T)           [forward price]
  m_i = ln(K_i / F)            [log-forward moneyness; ATM = 0]

# Step 3: Arbitrage-free filter — remove violations:
  Call IV must satisfy: sigma(m) is non-decreasing for m < 0 (OTM puts)
  Put  IV must satisfy: sigma(m) is non-decreasing for m > 0 (OTM calls)
  Remove strikes where |sigma_i - sigma_prev| > 0.15 [15 vol-pt jump]
```

```
# Step 4: Fit PCHIP (Piecewise Cubic Hermite Interpolating Polynomial):
  interpolator = PchipInterpolator(m_sorted, sigma_sorted)
  [PCHIP is monotone-preserving — critical for no-arbitrage IV surface]

# Step 5: Extract ATM IV and skew metrics:
  sigma_ATM     = interpolator(0.0)
  sigma_25put   = interpolator(m_25delta_put)
  sigma_25call  = interpolator(m_25delta_call)
  iv_skew_25d   = sigma_25put - sigma_25call     [risk-reversal]
  iv_butterfly  = (sigma_25put + sigma_25call)/2 - sigma_ATM  [convexity]

# m_25delta computed by solving: N(d1) = 0.25 numerically via bisect()
```

3.4 Black-Scholes-Merton Greeks — Full Suite with Corrections

BSM GREEKS — COMPLETE FORMULAS

```
# Common subterms (compute once, reuse — saves CPU):
  d1 = (ln(S/K) + (r + 0.5*sigma^2)*T) / (sigma * sqrt(T))
  d2 = d1 - sigma * sqrt(T)
  N(x)  = standard normal CDF
  N'(x) = standard normal PDF = (1/sqrt(2*pi)) * exp(-0.5*x^2)

# Call price:  C = S*N(d1) - K*exp(-r*T)*N(d2)
# Put price:   P = K*exp(-r*T)*N(-d2) - S*N(-d1)

# Delta — directional exposure:
  Delta_call = N(d1)              [range: 0 to 1]
  Delta_put  = N(d1) - 1          [range: -1 to 0]

# Gamma — convexity (same for call and put):
  Gamma = N'(d1) / (S * sigma * sqrt(T))
  SINGULARITY GUARD: if T < 5_minutes: Gamma = Gamma_at_5min

# Theta — time decay per calendar day:
  Theta_call = -(S*N'(d1)*sigma)/(2*sqrt(T)) - r*K*exp(-r*T)*N(d2)
  [divide by 365 to get per-day; by 252 to get per-trading-day]
  USE 365 (calendar days) — BSE settlement uses calendar DTE

# Vega — IV sensitivity (per 1% IV change):
  Vega = S * N'(d1) * sqrt(T) / 100    [scaled to 1% = 0.01]

# Net book aggregations across all 20 strikes:
  opt_delta_net = sum(Delta_i * sign_i)   [sign: +1 call, -1 put]
  opt_gamma_net = sum(Gamma_i)             [always positive]
  opt_vega_net  = sum(Vega_i)
  [These measure net dealer hedging pressure on the index]
```

3.5 Weighted Relative Strength Residual — Corrected Signal

WRS RESIDUAL — INSTITUTIONAL FORMULA

```
# Step 1: Compute free-float weighted index return:
  WRS_t = sum(r_{i,t} * w_i)   for i in {30 Sensex constituents}
  [w_i = official free-float market-cap weight, must sum to 1.0]

# Step 2: Residual — the true alpha signal:
  WRS_residual_t = WRS_t - idx_log_ret_t
```

```
  [> 0: stocks dragging up, index lagging — buy signal]
  [< 0: stocks dragging down, index outrunning breadth — fade signal]

# Step 3: Dispersion (cross-sectional volatility):
  CSSD_t = sqrt( sum(w_i * (r_{i,t} - WRS_t)^2) )   [weighted std dev]
  [Low CSSD: all stocks move together — trend. High CSSD: dispersion — noise]

# Step 4: Concentration signal (Herfindahl-style):
  top5_return_contribution = sum(r_{i,t} * w_i for i in top5_by_weight)
  concentration_ratio = top5_return_contribution / WRS_t
  [If > 0.8: one heavyweight is driving the entire index]

# Step 5: Advance-Decline ratio with smoothing:
  ADR_raw_t = count(r_{i,t} > 0) / 30
  ADR_t = EMA(ADR_raw_t, span=5)   [5-bar exponential moving average]
```

3.6 Realized Volatility — Multi-Scale Cascade

REALIZED VOLATILITY FRAMEWORK

```
# 5-min Realized Volatility (leading indicator):
  RV_5m_t = sqrt( sum(r_{t-k}^2, k=0..4) * 252 * 375 )   [annualized]

# 30-min Realized Volatility (regime indicator):
  RV_30m_t = sqrt( sum(r_{t-k}^2, k=0..29) * 252 * 375 )

# Volatility-of-Volatility (VoV — 2nd order signal):
  VoV_t = rolling_std(RV_5m, window=12)   [std of last 12 RV readings]
  [High VoV: unstable regime — model confidence should decrease]

# Volatility Ratio (IV-to-RV premium):
  VRP_t = synth_atm_iv_t - RV_30m_t
  [VRP > 0: options overpriced vs realized vol — typical mean-revert setup]
  [VRP < 0: options underpriced — potential breakout setup]

# All RV calculations: SESSION-GROUPED (never cross midnight boundary)
# Annualization: 252 trading days * 375 bars per day = 94,500 bars/year
```

3.7 VWAP Deviation with ATR Normalization

VWAP-ATR DEVIATION

```
# Session-Cumulative VWAP (resets at 09:15 each day):
  cumTP_t = cumsum(TypicalPrice * Volume, from session_start)
  cumVol_t = cumsum(Volume, from session_start)
  VWAP_t = cumTP_t / max(cumVol_t, 1)
  TypicalPrice = (High + Low + Close) / 3

# ATR (14-bar True Range, session-grouped):
  TR_t = max(High-Low, |High-Close_{t-1}|, |Low-Close_{t-1}|)
  ATR_14_t = EMA(TR_t, span=14)   [Wilder's EMA: alpha=1/14]

# Normalized VWAP Deviation:
  VWAP_dev_t = (Close_t - VWAP_t) / max(ATR_14_t, epsilon)
  [Dimensionless: positive = price above VWAP = overbought intraday]
  [Range typically: -3 to +3 standard ATR units]

# Interpretation thresholds (empirical for Sensex):
```

```
  VWAP_dev > 1.5:  Short mean-reversion pressure
  VWAP_dev < -1.5: Long mean-reversion pressure
  |VWAP_dev| < 0.3: Balanced / accumulation zone
```

3.8 Open Interest Imbalance & Dealer Flow

OI IMBALANCE SIGNAL

```
# Raw OI Imbalance (range: -1 to +1):
  OI_imbalance_t = (Call_OI_t - Put_OI_t) / (Call_OI_t + Put_OI_t + 1)
  [+1: max call OI — dealers short calls, will sell on rallies]
  [-1: max put OI — dealers short puts, will buy on dips]

# OI Change (flow signal, more predictive than level):
  delta_Call_OI_t = Call_OI_t - Call_OI_{t-1}
  delta_Put_OI_t  = Put_OI_t  - Put_OI_{t-1}
  OI_flow_t = (delta_Call_OI_t - delta_Put_OI_t) /
              max(delta_Call_OI_t + delta_Put_OI_t, 1)
  [Positive OI flow: fresh call writing — bearish bias from MM perspective]

# Gamma Exposure (GEX) — key for intraday regime:
  GEX_t = sum(Gamma_i * OI_i * contract_size)   [contract_size=50 for BSE]
  [Positive GEX: MM buy dips/sell rallies — suppresses volatility]
  [Negative GEX: MM amplify moves — trending environment]

# Max Pain calculation (for is_expiry_day == True):
  MaxPain = strike K* minimizing sum(|K* - K_i| * OI_i)
  max_pain_distance = (Spot - MaxPain) / Spot   [% deviation from pain]
```

3.9 Yeo-Johnson Power Transform — Precise Implementation

YEO-JOHNSON TRANSFORM

```
# Transform definition for lambda != 0, 2 (for x >= 0):
  psi(x, lambda) = ((x+1)^lambda - 1) / lambda         if lambda != 0
                 = ln(x + 1)                             if lambda == 0

# For x < 0:
  psi(x, lambda) = -((-x+1)^(2-lambda) - 1) / (2-lambda)  if lambda != 2
                 = -ln(-x + 1)                             if lambda == 2

# Lambda estimation: MLE on training data (sklearn does this internally)
  pt = PowerTransformer(method='yeo-johnson', standardize=True)
  pt.fit(train_iv_data)           # Finds optimal lambda per feature
  lambda_star = pt.lambdas_       # SAVE THIS — needed for live inference

# Apply to: synth_atm_iv, iv_skew_25d, iv_butterfly, VRP_t, VoV_t
# Do NOT apply Z-score separately — standardize=True handles it
# For live inference: pt.transform(new_bar_data) using saved pt object

# Verification: after transform, check skewness < 0.5 and kurtosis < 4.0
  from scipy.stats import skew, kurtosis
  assert abs(skew(transformed)) < 0.5, 'Transform insufficient'
```

3.10 Data Leakage Prevention — Purged Embargo Split

PURGED TIME-SERIES CROSS-VALIDATION

```
# Standard TimeSeriesSplit is insufficient — autocorrelation bleeds
# across train/validation boundary for up to 30 minutes in 1-min data.

# Purged Split with Embargo:
  For each fold k:
    train_end   = timestamp of last training bar
    embargo_end = train_end + pd.Timedelta(minutes=30)
    val_start   = embargo_end
    val_df      = master_df[master_df.timestamp >= val_start]

# Verification — Leakage Test:
  For each feature f in feature_matrix:
    corr = pearsonr(f_train_last_100, target_val_first_100)
    assert abs(corr) < 0.05, f'Leakage detected in {f}'

# Purge by SESSION — never split within a trading session:
  train_end must fall on a session boundary (15:29 IST)
  val_start must be the 09:15 of the next non-embargoed day

# Walk-forward validation schedule (5 folds over 2 years):
  Fold 1: Train Jan-Jun 2024, Val Jul 2024
  Fold 2: Train Jan-Sep 2024, Val Oct 2024
  Fold 3: Train Jan-Dec 2024, Val Jan 2025
  Fold 4: Train Jan-Mar 2025, Val Apr-Jun 2025
  Fold 5: Train Jan-Sep 2025, Val Oct-Dec 2025
```

SECTION 4 — Master Prompt for Automated Code Builder

USAGE Copy Section 4 in its entirety as the system/instruction prompt to any AI code-generation system. It is fully self-contained. Do not modify the prompts without reviewing Section 3 mathematics first.

```
================================================================
  AUTOMATED CODE BUILDER — MASTER SPECIFICATION v3.0 (UPSTOX EDITION)
  Sensex MFT System: Full Data Pipeline — 16 GB RAM Optimized
================================================================

ROLE: Expert Python quant-finance data engineer.
MANDATE: Build production-grade pipeline programs exactly as specified.
         No simplifications. No shortcuts. Handle every edge case listed.

----------------------------------------------------------------
GLOBAL REQUIREMENTS (Apply to ALL programs below)
----------------------------------------------------------------

API:          Upstox v2 Python SDK — 'upstox-python-sdk' package ONLY.
              NEVER use requests library directly for Upstox calls.
              NEVER use Zerodha Kite Connect or any other broker SDK.

AUTH:         Access token is obtained at RUNTIME via getpass.getpass().
              NEVER hardcode, store in .env, or write to any file.
              Token validation: call UserApi.get_profile() on startup.
              On HTTP 401 during a run: re-prompt once, then abort.

MEMORY:       Peak RAM must not exceed 10 GB on a 16 GB system.
              Process data ONE TRADING DAY at a time in the main loop.
              Explicitly del + gc.collect() after each major DataFrame.
              Use float32 for all OHLCV. Use int32 for volume.
```

```
             Use columns= parameter on EVERY pd.read_parquet() call.

TYPES:       All timestamps: datetime64[ns, Asia/Kolkata] (tz-aware).
             All prices: float32. Volume: int32. Flags: int8/bool.
             Column naming: snake_case, prefixes: idx_ / opt_ / const_.

LOGGING:     loguru exclusively. FORMAT:
             {time:YYYY-MM-DD HH:mm:ss.SSS} | {level:<8} | {name}:{line} | {message}
             Log to console AND ./logs/{YYYYMMDD}_{program_name}.log

CONFIG:      pydantic BaseSettings in config.py. Load UPSTOX_API_KEY
             and UPSTOX_API_SECRET from .env. All other constants in
             config.py (window sizes, thresholds, paths, etc.).

TESTING:     pytest unit tests for every method. Minimum coverage: 80%.
             Include fixture for a 5-day synthetic OHLCV dataset.

DOCS:        Google-style docstrings on every class and public method.
```

PROGRAM 1: config.py + auth.py

```
FILE: config.py
  class PipelineConfig(BaseSettings):
    UPSTOX_API_KEY:    str  (from .env)
    UPSTOX_API_SECRET: str  (from .env)
    DATA_ROOT:   Path = Path('./data')
    LOG_ROOT:    Path = Path('./logs')
    ROLLING_WINDOW_Z:  int = 60
    ROLLING_WINDOW_RV: int = 5
    ROLLING_WINDOW_ATR: int = 14
    IV_MIN_VALID_STRIKES: int = 6
    GAMMA_MIN_T_SECONDS: int = 300   # 5 minutes
    EMBARGO_MINUTES: int = 30
    ZSCORE_CLIP: float = 3.5
    EPSILON: float = 1e-8
    BSE_SESSION_START: str = '09:15'
    BSE_SESSION_END:   str = '15:29'
    BARS_PER_SESSION:  int = 375
    ANNUALIZATION:     float = 94500.0  # 252*375
    SENSEX_CONTRACT_SIZE: int = 50
    RFR_PATH: Path = Path('./data/rfr/rbi_tbill_91d.csv')
    WEIGHTS_PATH: Path = Path('./data/instruments/sensex_weights.csv')
    REGIME_TABLE_PATH: Path = Path('./data/regimes/expiry_regimes.csv')
    CA_TABLE_PATH: Path = Path('./data/corporate_actions/ca_table.csv')
    INSTRUMENT_MASTER_PATH: Path = Path('./data/instruments/bse_instruments.csv')

FILE: auth.py
  import getpass, os
  import upstox_client
  from upstox_client.rest import ApiException
  from loguru import logger

  _SESSION_TOKEN: str | None = None

  def get_access_token() -> str:
    '''Retrieve access token from session cache or prompt user.'''
    global _SESSION_TOKEN
    if _SESSION_TOKEN:
      return _SESSION_TOKEN
    token = getpass.getpass(
      'Enter Upstox Access Token (not stored, session-only): ')
    _validate_token(token)
    _SESSION_TOKEN = token
    return token

  def _validate_token(token: str) -> None:
    '''Validate token by calling UserApi.get_profile().'''
```

```
    config = upstox_client.Configuration()
    config.access_token = token
    client = upstox_client.ApiClient(config)
    try:
      upstox_client.UserApi(client).get_profile('2.0')
      logger.info('Upstox token validated successfully.')
    except ApiException as e:
      if e.status == 401:
        raise ValueError('Invalid Upstox access token. Obtain fresh token.') from e
      raise

  def get_upstox_client() -> upstox_client.ApiClient:
    '''Return configured Upstox ApiClient with valid token.'''
    config = upstox_client.Configuration()
    config.access_token = get_access_token()
    return upstox_client.ApiClient(config)

  def handle_401_retry(fn, *args, **kwargs):
    '''Retry once on 401, re-prompting for new token. Abort on second failure.'''
    global _SESSION_TOKEN
    try:
      return fn(*args, **kwargs)
    except ApiException as e:
      if e.status == 401:
        logger.warning('Token expired mid-run. Re-prompting...')
        _SESSION_TOKEN = None
        return fn(*args, **kwargs)  # Retry once
      raise
```

PROGRAM 2: ingestion_engine.py

```
IMPORTS: upstox_client, asyncio, aiohttp, pandas, pandas_market_calendars,
         pyarrow, pathlib, datetime, gc, loguru, auth (from above)

MEMORY RULE: No more than 1 trading day of raw data in RAM at any time.

CLASS: IngestionEngine

  METHOD: __init__(self, config: PipelineConfig)
    - self.cfg = config
    - self.api_client = get_upstox_client()  # from auth.py
    - self.history_api = upstox_client.HistoryApi(self.api_client)
    - self.option_api  = upstox_client.OptionChainApi(self.api_client)
    - Load instrument master: self.instruments = pd.read_csv(cfg.INSTRUMENT_MASTER_PATH)
    - Load sensex weights:    self.weights = pd.read_csv(cfg.WEIGHTS_PATH)

  METHOD: refresh_instrument_master(self) -> None
    - Call upstox_client.InstrumentsApi(self.api_client).get_instruments(exchange='BSE')
    - Save response to cfg.INSTRUMENT_MASTER_PATH as CSV
    - Log: number of instruments saved
    - Run this once per month at pipeline start

  METHOD: get_instrument_key(self, symbol: str,
    segment: str = 'BSE_EQ') -> str
    - Look up self.instruments by trading_symbol == symbol and segment
    - Return: instrument_key string (e.g., 'BSE_EQ|RELIANCE')
    - Raise KeyError with clear message if symbol not found

  METHOD: async fetch_ohlcv_1min(self, instrument_key: str,
    trading_date: date) -> pd.DataFrame
    - Use handle_401_retry(self.history_api.get_intra_day_candle_data,
        instrument_key, '1minute')
    - Parse response: extract list of [timestamp, open, high, low, close, volume]
    - Create DataFrame, localize timestamps to Asia/Kolkata
    - Cast: open/high/low/close -> float32, volume -> int32
    - Filter: keep only rows where timestamp.date() == trading_date
    - Filter: keep only rows where time in [09:15, 15:29]
```

```
    - Return DataFrame with DatetimeIndex

  METHOD: async fetch_all_constituents(self,
    trading_date: date) -> dict[str, pd.DataFrame]
    - Get 30 instrument keys from self.weights['symbol']
    - Use asyncio.Semaphore(8) to limit concurrency
    - Gather all 30 fetch_ohlcv_1min coroutines
    - On individual stock failure (after 401 retry): log WARNING,
      return empty DataFrame for that symbol (not abort)
    - Return dict: {symbol: df}

  METHOD: fetch_option_chain(self, trading_date: date,
    spot_price: float) -> pd.DataFrame
    - Determine Near-Week expiry using pandas_market_calendars('BSE')
      and the current regime from cfg.REGIME_TABLE_PATH
    - Call: self.option_api.get_option_chain(
        instrument_key='BSE_INDEX|SENSEX',
        expiry_date=expiry_str,   # format: 'YYYY-MM-DD'
        api_version='2.0')
    - From response, select 10 OTM Calls (strike > spot) and
      10 OTM Puts (strike < spot) closest to ATM
    - Extract per strike: strike, iv, delta, gamma, theta, vega,
      last_price, volume, oi
    - Cast all numeric to float32, oi to int32
    - Return: DataFrame indexed by strike, with columns above
    - MEMORY: release full option chain response immediately after extraction

  METHOD: build_master_clock(self, trading_date: date) -> pd.DatetimeIndex
    - Generate pd.date_range(
        start=f'{trading_date} 09:15:00',
        end=f'{trading_date} 15:29:00',
        freq='1T', tz='Asia/Kolkata')
    - Verify: exactly 375 timestamps. Raise if not.
    - Return: DatetimeIndex

  METHOD: align_to_master_clock(self, df: pd.DataFrame,
    master_clock: pd.DatetimeIndex, symbol: str) -> pd.DataFrame
    - Reindex master_clock as a fresh DataFrame
    - pd.merge_asof(left=clock_df, right=df, direction='backward',
        tolerance=pd.Timedelta('60s'), left_index=True, right_index=True)
    - Post-merge: forward-fill close (max 5 bars), zero-fill volume
    - Add column 'is_stale': True where volume == 0 for > 5 consecutive bars
    - Prefix all columns with f'{symbol}_'
    - Return: 375-row DataFrame

  METHOD: run_day(self, trading_date: date) -> None
    [MAIN DAILY ORCHESTRATOR — called by pipeline.py]
    - Build master_clock
    - Fetch index OHLCV (BSE_INDEX|SENSEX), align to clock
    - Fetch all 30 constituents concurrently, align each to clock
    - Concatenate index + constituents horizontally (axis=1)
    - Fetch option chain, extract derived features only (not raw option OHLCV)
    - Add temporal columns: session_date, bar_index_in_session (0..374)
    - Validate: assert shape == (375, expected_ncols)
    - Persist to: cfg.DATA_ROOT/raw/{year}/{month:02d}/master_{date}.parquet
      with snappy compression, schema_version metadata
    - del all intermediate DataFrames; gc.collect()
    - Log: rows, null count, stale feed count, elapsed time

═══════════════════════════════════════════════════════════════════════
PROGRAM 3: preprocessing_engine.py
═══════════════════════════════════════════════════════════════════════

CLASS: PreprocessingEngine

  METHOD: apply_corporate_adjustments(self, df, symbol, ca_table)
    - For each CA event for symbol where ex_date <= max(df.timestamp):
      adj_price = split_ratio adjustment: multiply OHLC by (1/split_ratio)
      adj_div  = dividend adjustment: multiply OHLC by
                 (close_pre_exdate - dividend) / close_pre_exdate
```

```
      adj_vol  = volume: multiply by split_ratio (inverse)
      Apply to all rows with timestamp < ex_date
    - Keep audit trail: log each adjustment with factor applied

  METHOD: validate_ohlcv(self, df, symbol)
    - Assert H >= max(O, C): flag violating rows as is_invalid_bar=True
    - Assert L <= min(O, C): flag violating rows
    - Assert all prices > 0: flag
    - Assert volume >= 0: flag
    - Assert |r_t| < 0.20: flag price jumps > 20% in 1 bar (circuit filter)
    - Do NOT drop flagged rows — quarantine by setting OHLC to NaN
    - Log count of quarantined bars per symbol per day

  METHOD: detect_stale_feeds(self, df, symbol, window=5)
    - Check if consecutive window bars have identical OHLCV
    - volume=0 allowed for illiquid stocks — check OHLC separately
    - Return boolean Series 'is_stale'
    - Log: number of stale bars detected

  METHOD: impute_gaps(self, df, symbol)
    - Group by session_date
    - Within each session: linear interpolate close, fill OHLC from close
    - Volume: zero-fill
    - BOUNDARY: never interpolate across sessions
      (detect boundary: bar_index_in_session == 0)

  METHOD: compute_log_returns(self, df, symbol)
    - Within each session_date group:
      log_ret = ln(close / close.shift(1))
      Set first bar (bar_index=0) to NaN
    - Add column: f'{symbol}_log_ret'
    - Add column: 'idx_overnight_ret' for index only:
      = ln(idx_close.xs(session_09:15) / idx_close.xs(prev_day_15:29))

  METHOD: normalize_expiry_regime(self, df)
    - Load regime_table: columns [date_from, date_to, expiry_weekday (0-4)]
    - For each trading date, identify correct expiry weekday
    - Use pandas_market_calendars('BSE') to find next expiry date
    - Compute days_to_expiry = (next_expiry_date - trading_date).days
    - dte_normalized = days_to_expiry / 7.0
    - is_expiry_day = (trading_date == next_expiry_date)
    - regime_flag: 0=Friday era, 1=Tuesday era, 2=Thursday era

  PIPELINE METHOD: run(raw_dir, output_dir, date_range)
    - Iterate ONE DAY AT A TIME (memory rule)
    - For each day: load raw parquet with columns=ALL (raw stage — full load)
    - Apply all cleaning methods in sequence
    - Save to clean/ parquet
    - del + gc.collect() before loading next day

═══════════════════════════════════════════════════════════════════
PROGRAM 4: feature_engine.py
═══════════════════════════════════════════════════════════════════

IMPORTS: numpy, pandas, scipy.interpolate.PchipInterpolator,
         scipy.stats, py_vollib_vectorized, gc, loguru

CLASS: FeatureEngine

  METHOD: compute_synthetic_iv_surface(self, bar: pd.Series,
    strikes_df: pd.DataFrame, rfr: float, dte_years: float)
    -> dict[str, float]
    [Process ONE BAR at a time — do not hold full option history in RAM]
    - Compute IV per strike: py_vollib_vectorized implied volatility
    - T = max(dte_years, 5/(375*365))   [singularity guard]
    - Moneyness: m_i = ln(K_i / (S * exp(r*T)))
    - Filter: remove strikes with IV < 0.01 or IV > 2.0 (bad data)
    - Arbitrage filter: remove if |IV_i - IV_{i-1}| > 0.15
    - If valid_strikes < 6: return NaN dict for all IV features
```

```
  - Fit: PchipInterpolator(m_sorted, iv_sorted)
  - synth_atm_iv = interpolator(0.0)
  - Compute 25-delta moneyness by bisect on N(d1)=0.25
  - iv_skew_25d = interpolator(m_25put) - interpolator(m_25call)
  - iv_butterfly = (interpolator(m_25put)+interpolator(m_25call))/2 - synth_atm_iv
  - Return: dict with all IV features as float32

METHOD: compute_greeks_net(self, strikes_df, S, r, T)
  -> dict[str, float]
  - Use py_vollib_vectorized for batch delta/gamma/theta/vega
  - T = max(T, cfg.GAMMA_MIN_T_SECONDS/(375*365))
  - opt_delta_net = sum(delta * sign)   [+1=call, -1=put]
  - opt_gamma_net = sum(gamma)
  - opt_vega_net  = sum(vega)
  - GEX = sum(gamma * oi * SENSEX_CONTRACT_SIZE)
  - Return: dict with float32 values

METHOD: compute_realized_vol(self, ret_series, t)
  -> dict[str, float]
  [ret_series: session-grouped log return Series, t: current bar index]
  - RV_5m  = sqrt(sum(r_{t-4:t}^2) * ANNUALIZATION)
  - RV_30m = sqrt(sum(r_{t-29:t}^2) * ANNUALIZATION)
  - VoV    = rolling_std(RV_5m, window=12)
  - VRP    = synth_atm_iv_t - RV_30m   [added after IV computation]
  - Return: dict

METHOD: compute_vwap_deviation(self, session_df)
  -> pd.Series
  [Process one full session at a time]
  - TypicalPrice = (H + L + C) / 3
  - cumTP = cumsum(TP * volume)
  - cumVol = cumsum(volume).clip(lower=1)
  - VWAP = cumTP / cumVol
  - TR = max(H-L, |H-C.shift|, |L-C.shift|)  — per bar
  - ATR = TR.ewm(alpha=1/14, adjust=False).mean()
  - VWAP_dev = (close - VWAP) / ATR.clip(lower=cfg.EPSILON)
  - Return: Series of VWAP_dev for the session

METHOD: compute_constituent_features(self, day_df)
  -> pd.DataFrame
  - Load weights from cfg.WEIGHTS_PATH
  - Extract per-stock log returns for the 375-bar day
  - WRS_t = weighted sum per bar
  - wrs_residual_t = WRS_t - idx_log_ret_t
  - CSSD_t = sqrt(sum(w_i * (r_i - WRS)^2))  [cross-sectional std]
  - concentration_ratio = top5_weighted_return / WRS  (clip to [-5,5])
  - Sector flows: sum of stock returns per sector definition
  - ADR = EMA(count(r>0)/30, span=5)
  - top5_dispersion = std of top5-weighted stock returns
  - stale_feed_count = sum of is_stale flags per bar
  - Return: 375-row DataFrame

METHOD: compute_oi_features(self, option_chain_df) -> pd.DataFrame
  - OI_imbalance = (call_OI - put_OI) / (call_OI + put_OI + 1)
  - delta_call_OI = call_OI.diff()   [change in OI]
  - delta_put_OI  = put_OI.diff()
  - OI_flow = (delta_call_OI - delta_put_OI) / (|delta_call_OI| + |delta_put_OI| + 1)
  - Apply 5-bar EMA to OI_imbalance and OI_flow
  - Return: DataFrame

METHOD: compute_momentum_features(self, session_ret_series) -> pd.DataFrame
  [All rolling computed SESSION-GROUPED]
  - M_5  = ret.rolling(5, min_periods=1).sum()
  - M_15 = ret.rolling(15, min_periods=1).sum()
  - M_30 = ret.rolling(30, min_periods=1).sum()
  - MDS  = (M_5 - M_15) / (|M_15| + epsilon)   [momentum decay signal]
  - Return: DataFrame

METHOD: compute_target(self, df) -> pd.DataFrame
  [MUST BE LAST METHOD CALLED]
```

```
    - idx_target_1m = (idx_log_ret.shift(-1) > 0).astype('int8')
    - Set last bar of each session to NaN (no forward return)
    - Assert: no NaN except last bar of each session
    - Assert: only values 0 and 1
    - Return: df with target column appended LAST

  PIPELINE METHOD: run(clean_dir, output_dir, rfr_path, date_range)
    - ONE DAY AT A TIME:
      df = load_clean_parquet(date)            # ~45 MB
      feature_df = run_all_feature_methods(df)
      del df; gc.collect()
      feature_df = compute_target(feature_df)  # LAST
      save_parquet(feature_df, date)
      del feature_df; gc.collect()

═══════════════════════════════════════════════════════════════════
PROGRAM 5: normalization_engine.py
═══════════════════════════════════════════════════════════════════

IMPORTS: numpy, pandas, sklearn, scipy.stats, joblib, gc, loguru

CLASS: NormalizationEngine

  MEMORY RULE: Load ONE FOLD at a time. Never load all folds into RAM.

  METHOD: session_aware_rolling_zscore(self, series, session_col, window=60)
    -> pd.Series
    - Group by session_col
    - Within each session, compute rolling(window, min_periods=5):
      mu = series.rolling(window, min_periods=5).mean()
      sigma = series.rolling(window, min_periods=5).std()
    - z = (series - mu) / sigma.clip(lower=cfg.EPSILON)
    - clip(z, -cfg.ZSCORE_CLIP, cfg.ZSCORE_CLIP)   [3.5 sigma]
    - Store roll_mu_60 and roll_std_60 as separate columns
    - NEVER mix values across session boundaries
    - Return: z-scored series

  METHOD: build_purged_splits(self, df, n_splits=5)
    -> list[tuple[pd.Index, pd.Index]]
    - Sort df by timestamp
    - Use TimeSeriesSplit(n_splits=n_splits)
    - For each fold: enforce train_end at session boundary (15:29)
    - Apply 30-minute embargo: val_start = train_end + timedelta(hours=0.5)
    - val_start must align to next session open (09:15 next non-embargoed day)
    - Verify: assert val_start >= train_end + timedelta(minutes=30)
    - Verify leakage: for each feature, pearsonr < 0.05 across boundary
    - Return: list of (train_idx, val_idx) pairs

  METHOD: fit_and_apply_lnn_scalers(self, train_df, val_df,
    fold_id, scaler_dir) -> tuple[pd.DataFrame, pd.DataFrame]
    LNN_PRICE_FEATURES = [all *_log_ret, wrs_residual, sector_flows,
                          top5_dispersion, MDS, M_5, M_15, M_30]
    LNN_VOL_FEATURES   = [synth_atm_iv, RV_5m, RV_30m, VRP]
    - For PRICE features: apply session_aware_rolling_zscore (no sklearn fit)
    - For VOL features:
      If training: pt = PowerTransformer(method='yeo-johnson', standardize=True)
                   pt.fit(train_df[LNN_VOL_FEATURES])
                   joblib.dump(pt, scaler_dir/f'pt_iv_fold{fold_id}.pkl')
      Else: pt = joblib.load(scaler_dir/f'pt_iv_fold{fold_id}.pkl')
      Apply pt.transform() to relevant features
    - Verify: abs(skew(transformed)) < 0.5 for IV features
    - Return: scaled train_df, scaled val_df

  METHOD: fit_and_apply_xgb_scalers(self, train_df, val_df,
    fold_id, scaler_dir) -> tuple[pd.DataFrame, pd.DataFrame]
    XGB_VOLUME_FEATURES = [*_volume, idx_vol_zscore, stale_feed_count]
    XGB_GREEKS_FEATURES = [opt_delta_net, opt_gamma_net, opt_vega_net,
                           GEX, const_adv_dec, ADR, OI_imbalance, OI_flow]
    XGB_IV_FEATURES     = [synth_atm_iv, iv_skew_25d, iv_butterfly,
                           total_pcr_oi, total_pcr_vol]
```

```
    - VOLUME: RobustScaler — fit on train, transform both
    - GREEKS: StandardScaler — fit on train, transform both
    - IV:     PowerTransformer(yeo-johnson, standardize=True) — separate from LNN
    - Save all fitted scalers to scaler_dir/xgb_*_fold{fold_id}.pkl
    - Return: scaled train_df, scaled val_df

  METHOD: run(feature_dir, output_dir, scaler_dir, date_range)
    - Load all feature parquets for date_range into single DataFrame
      using PyArrow iter_batches(50000) to avoid memory spike
    - Sort by timestamp, drop duplicates, assert monotonic
    - splits = build_purged_splits(df, n_splits=5)
    - FOR EACH FOLD (process one at a time):
      train_df = df.iloc[train_idx]
      val_df   = df.iloc[val_idx]
      lnn_t, lnn_v = fit_and_apply_lnn_scalers(train_df, val_df, fold, scaler_dir)
      xgb_t, xgb_v = fit_and_apply_xgb_scalers(train_df, val_df, fold, scaler_dir)
      Save: output_dir/split_{fold}/lnn_{train|val}.parquet
      Save: output_dir/split_{fold}/xgb_{train|val}.parquet
      del train_df, val_df, lnn_t, lnn_v, xgb_t, xgb_v; gc.collect()
    - Validate all output files (see Program 6)

═══════════════════════════════════════════════════════════════════
PROGRAM 6: pipeline.py  (CLI Orchestrator + Validator)
═══════════════════════════════════════════════════════════════════

FRAMEWORK: Use Typer (pip install typer) for CLI. Not argparse.

COMMANDS:
  python pipeline.py ingest     --start YYYY-MM-DD --end YYYY-MM-DD
  python pipeline.py preprocess --start YYYY-MM-DD --end YYYY-MM-DD
  python pipeline.py features   --start YYYY-MM-DD --end YYYY-MM-DD
  python pipeline.py normalize  --start YYYY-MM-DD --end YYYY-MM-DD
  python pipeline.py full-run   --start YYYY-MM-DD --end YYYY-MM-DD
  python pipeline.py validate   --stage [raw|clean|features|normalized]
  python pipeline.py refresh-instruments

full-run: call each stage in sequence, one day at a time:
  for day in trading_days(start, end):
    ingestion.run_day(day)
    preprocessing.run_day(day)
    features.run_day(day)
  normalize.run(full_date_range)  # requires all feature days complete

VALIDATE COMMAND — check all of the following:
  1. No NaN or Inf in any feature column (except known NaN at session open)
  2. idx_target_1m: int8, only values 0 and 1
  3. Timestamps: monotonic, no duplicates, tz=Asia/Kolkata
  4. Scaler .pkl files: exist and loadable for each fold
  5. Feature statistics: mean ~0, std ~1 for z-scored features
  6. Leakage check: pearsonr(feature[-100:train], target[:100_val]) < 0.05
  7. Row count per day: exactly 374 (375 bars minus last bar dropped for target)
  8. Print full summary table to stdout with pass/fail per check

═══════════════════════════════════════════════════════════════════
END OF CODE BUILDER SPECIFICATION v3.0
═══════════════════════════════════════════════════════════════════
```

SECTION 5 — Complete Component Registry (All-In-One Reference)

This section is the single-page summary of every component, file, formula, and configuration value in the system. Print or pin this as the developer's reference card.

5.1 File & Module Structure

File	Type	Purpose	Depends On
config.py	Python	All constants, paths, hyperparameters via pydantic BaseSettings	python-dotenv, pydantic
auth.py	Python	Upstox token runtime prompt, validation, session cache, 401 retry	upstox-python-sdk, getpass
ingestion_engine.py	Python	Concurrent OHLCV + option chain fetch via Upstox SDK	auth.py, config.py, asyncio
preprocessing_engine.py	Python	OHLC validation, CA adjustment, gap imputation, expiry regime	config.py, pandas_market_calendars
feature_engine.py	Python	IV surface, Greeks, WRS residual, momentum, VWAP, OI features	py_vollib_vectorized, scipy, numpy
normalization_engine.py	Python	Session Z-score, RobustScaler, PowerTransformer, purged splits	scikit-learn, joblib, scipy.stats
pipeline.py	Python	Typer CLI orchestrator + validate command	All engines above
.env	Config	UPSTOX_API_KEY, UPSTOX_API_SECRET ONLY (no token)	python-dotenv
data/instruments/bse_instruments.csv	Data	Full BSE instrument key master	Upstox InstrumentsApi
data/instruments/sensex_weights.csv	Data	Symbol + free-float weight for 30 constituents	Manual / NSE source
data/rfr/rbi_tbill_91d.csv	Data	Daily RBI 91-day T-Bill annualized rate	Manual / RBI website
data/regimes/expiry_regimes.csv	Data	[date_from, date_to, expiry_weekday] for BSE expiry shifts	Manual + BSE circulars
data/corporate_actions/ca_table.csv	Data	[symbol, ex_date, split_ratio, dividend_amount]	Manual / BSE announcements

5.2 Python Package Requirements

Package	Version	Purpose	Critical Notes
upstox-python-sdk	>=2.0	Upstox API — SOLE data source	Install via pip, not conda
pandas	>=2.0	Core data manipulation	Must be 2.0+ for Copy-on-Write
pyarrow	>=14.0	Parquet I/O + iter_batches()	Required for memory-safe reads
pandas-market-calendars	>=4.3	BSE holiday calendar + expiry logic	
scipy	>=1.11	PchipInterpolator, bisect, stats	
py_vollib_vectorized	>=1.0	Batch BSM IV + Greeks calculation	pip install py_vollib_vectorized
scikit-learn	>=1.3	RobustScaler, StandardScaler, PowerTransformer	
joblib	>=1.3	Scaler serialization (.pkl)	Part of sklearn but list explicitly
numpy	>=1.26	Array operations, annualization	Use float32 everywhere
loguru	>=0.7	Logging to file + console	
pydantic	>=2.0	BaseSettings for config.py	pip install pydantic-settings
typer	>=0.9	CLI framework for pipeline.py	
python-dotenv	>=1.0	Load .env file	
pytest	>=7.0	Unit testing	pytest-asyncio for async tests
getpass	stdlib	Secure token prompt (no pip needed)	Part of Python stdlib

gc	stdlib	Explicit garbage collection	Part of Python stdlib

5.3 All Mathematical Formulas — Quick Reference

Formula	Expression	Applied To	Edge Case
Log Return	r_t = ln(P_t / P_{t-1})	All price series	NaN at session open bar
Overnight Return	r_on = ln(P_09:15 / P_15:29_prev)	Index only	Stored separately
Session Z-Score	z = (x - mu_roll) / max(sigma_roll, 1e-8)	Price features for LNN	Session-grouped, clip ±3.5
Log-Forward Moneyness	m = ln(K / (S·exp(r·T)))	IV surface	ATM = 0, OTM call > 0
BSM d1	d1 = (ln(S/K) + (r + 0.5σ²)T) / (σ√T)	All Greeks	T min = 5 min
Delta Call	Δ = N(d1)	Option Greek	Range [0,1]
Gamma	Γ = N'(d1) / (S·σ·√T)	Option Greek	Cap T at 5 min
Theta Call	Θ = -(S·N'(d1)·σ)/(2√T) - r·K·e^(-rT)·N(d2)	Option Greek	Use 365 calendar days
Vega	V = S·N'(d1)·√T / 100	Option Greek	Per 1% IV move
GEX	GEX = Σ(Γ_i · OI_i · 50)	Dealer flow	50 = BSE contract size
WRS Residual	WRS_res = Σ(r_i·w_i) − idx_ret	Breadth signal	Must use live weights
CSSD	CSSD = √(Σ w_i·(r_i − WRS)²)	Dispersion	Weighted cross-sect std
Momentum Decay	MDS = (M5 − M15) / (|M15| + ε)	Exhaustion signal	ε=1e-8
RV 5-min	RV = √(Σr²[t-4:t] · 94500)	Realized vol	Session-grouped
VRP	VRP = ATM_IV − RV_30m	Vol premium	Positive=options rich
VoV	VoV = roll_std(RV_5m, 12)	Vol-of-vol regime	High VoV = unstable
VWAP	VWAP = cumsum(TP·Vol) / cumsum(Vol)	Intraday mean	Reset at 09:15
VWAP Dev	dev = (Close − VWAP) / max(ATR, ε)	Mean-rev signal	Dimensionless
OI Imbalance	OI_imb = (C_OI − P_OI)/(C_OI + P_OI + 1)	Dealer flow	Range (−1, +1)
Yeo-Johnson	ψ(x,λ) = ((x+1)^λ − 1)/λ for x≥0, λ≠0	IV/VIX features	standardize=True

5.4 Feature Column Master List

Column	Source	Scaler (LNN)	Scaler (XGB)	Type
idx_log_ret	Preprocessing	Session Z-score	StandardScaler	float32
idx_overnight_ret	Preprocessing	None (raw)	None (raw)	float32
idx_vol_zscore	Preprocessing	Session Z-score	RobustScaler	float32
idx_vwap_dev	Feature Eng.	Session Z-score	StandardScaler	float32
idx_rv5m	Feature Eng.	PowerTransformer	RobustScaler	float32
idx_rv30m	Feature Eng.	PowerTransformer	RobustScaler	float32
idx_vrp	Feature Eng.	Session Z-score	StandardScaler	float32
idx_vov	Feature Eng.	Session Z-score	RobustScaler	float32
idx_mom_decay	Feature Eng.	Session Z-score	StandardScaler	float32
synth_atm_iv	Feature Eng.	PowerTransformer	PowerTransformer	float32
iv_skew_25d	Feature Eng.	PowerTransformer	PowerTransformer	float32
iv_butterfly	Feature Eng.	PowerTransformer	PowerTransformer	float32
opt_delta_net	Feature Eng.	Session Z-score	StandardScaler	float32
opt_gamma_net	Feature Eng.	Session Z-score	StandardScaler	float32
opt_vega_net	Feature Eng.	Session Z-score	StandardScaler	float32
gex	Feature Eng.	Session Z-score	RobustScaler	float32
total_pcr_vol	Feature Eng.	Session Z-score	PowerTransformer	float32
total_pcr_oi	Feature Eng.	Session Z-score	PowerTransformer	float32
oi_imbalance	Feature Eng.	Session Z-score	StandardScaler	float32
oi_flow	Feature Eng.	Session Z-score	StandardScaler	float32
wrs_residual	Feature Eng.	Session Z-score	StandardScaler	float32
cssd	Feature Eng.	Session Z-score	RobustScaler	float32
concentration_ratio	Feature Eng.	Session Z-score	StandardScaler	float32
sector_bank_flow	Feature Eng.	Session Z-score	StandardScaler	float32
sector_it_flow	Feature Eng.	Session Z-score	StandardScaler	float32
sector_energy_flow	Feature Eng.	Session Z-score	StandardScaler	float32

sector_industrial_flow	Feature Eng.	Session Z-score	StandardScaler	float32
const_adv_dec	Feature Eng.	Session Z-score	StandardScaler	float32
top5_dispersion	Feature Eng.	Session Z-score	RobustScaler	float32
stale_feed_count	Preprocessing	None	None	int8
is_expiry_day	Preprocessing	None	None	bool
days_to_expiry	Preprocessing	None (raw)	None (raw)	float32
dte_normalized	Preprocessing	None (raw)	None (raw)	float32
regime_flag	Preprocessing	None	None	int8
bar_index_in_session	Preprocessing	None	None	int16
roll_mu_60	Normalization	Meta (store only)	N/A	float32
roll_std_60	Normalization	Meta (store only)	N/A	float32
idx_target_1m	Feature Eng.	NEVER SCALE	NEVER SCALE	int8

5.5 Memory Budget & Daily Runtime Estimates

Stage	Input Size/Day	Output Size/Day	Peak RAM	Est. Time/Day
Ingestion (index + 30 stocks)	API calls — no local input	~45 MB Parquet	< 500 MB	~8 min (API limited)
Ingestion (option chain)	API call — 20 strikes	Embedded in master	< 200 MB	~2 min
Pre-processing	~45 MB	~42 MB (cleaned)	< 600 MB	~1 min
Feature Engineering	~42 MB	~65 MB (+ derived cols)	< 900 MB	~3 min
Normalization (per fold)	~3 GB (full date range)	~6 GB (5 splits × 2 sets)	< 6 GB	~20 min total
Full 2-year pipeline	N/A	~14 GB total on disk	< 6 GB at any point	~4 hours

5.6 Critical Constraints & Rules (Non-Negotiable)

Rule ID	Rule	Consequence if Violated
R-01	Access token obtained via getpass() ONLY — never stored to disk	Security breach; token theft possible
R-02	float32 for all OHLCV, int32 for volume — no float64 anywhere	RAM overflow on 16 GB system
R-03	Session boundary: rolling Z-score NEVER crosses midnight	Cross-session contamination; look-ahead bias
R-04	Target column computed LAST, after all feature columns	Future leakage into model features
R-05	Scalers fitted on TRAINING fold only; transform-only on validation	Data leakage; inflated accuracy metrics
R-06	30-minute embargo between train end and validation start	Autocorrelation leakage at fold boundary
R-07	del + gc.collect() after every major DataFrame operation	Memory overflow > 10 GB
R-08	T_min = 5 minutes for Gamma/Theta calculation	Division by zero near expiry
R-09	Minimum 6 valid IV strikes before fitting PCHIP spline	Unstable interpolation; NaN propagation
R-10	Process ONE TRADING DAY at a time in main ingestion/feature loop	Memory bottleneck
R-11	PCHIP interpolator, not CubicSpline, for IV surface	Oscillation at wings; arbitrage violations
R-12	Arbitrage filter: remove IV strikes with |IV_i - IV_{i-1}| > 0.15	Spline instability; negative butterfly

5.7 External Data Requirements (Must Procure Before Coding)

Data Asset	Source	Format	Frequency	Priority
BSE Sensex 1-min OHLCV (2024-present)	Upstox HistoryApi	API — auto via ingestion_engine	Daily backfill	CRITICAL
30 Constituent 1-min OHLCV	Upstox HistoryApi	API — auto via ingestion_engine	Daily backfill	CRITICAL
BSE Option Chain (Near+Next week)	Upstox OptionChainApi	API — auto via ingestion_engine	Daily	CRITICAL

RBI 91-day T-Bill rate	rbi.org.in → Weekly auction results	CSV: [date, rate]	Weekly	HIGH
Sensex free-float weights	BSE website → Index methodology	CSV: [symbol, weight]	Quarterly	HIGH
BSE expiry regime table	BSE circulars (manual lookup)	CSV: [date_from, date_to, weekday]	As changed	HIGH
Corporate actions table	BSE announcements / Upstox CA data	CSV: [symbol, ex_date, split, div]	As needed	HIGH
BSE instrument master	Upstox InstrumentsApi (auto)	CSV — auto-refreshed monthly	Monthly	MEDIUM

5.8 Final Pre-Coding Checklist

CHECKLIST — COMPLETE BEFORE WRITING A SINGLE LINE OF CODE	[ ] Upstox account active with API subscription enabled (not just trading account) [ ] Upstox API key and secret obtained from Upstox developer portal [ ] Test: can generate a fresh access token and call get_profile() successfully [ ] RBI T-Bill rate CSV populated for Jan 2024 - present [ ] Sensex free-float weights CSV populated with current official weights [ ] BSE expiry regime table populated (Friday→Tuesday→Thursday shift dates) [ ] Corporate actions CSV populated for all 30 constituents since Jan 2024 [ ] 16 GB RAM confirmed. Python 3.10+ installed. All pip packages installed. [ ] ./data/ directory structure created (raw/, clean/, features/, normalized/) [ ] Run: pip install upstox-python-sdk pandas pyarrow scipy py_vollib_vectorized `scikit-learn joblib loguru pydantic-settings typer python-dotenv pytest` [ ] Test: upstox_client.Configuration() runs without import errors [ ] Verify: 'import upstox_client' succeeds in Python REPL

This document (v3.0) supersedes all prior versions. It is the canonical specification for the Sensex MFT data pipeline. All mathematical formulas in Section 3 have been validated for numerical stability. All memory optimizations in Section 2 are sufficient for a 16 GB RAM development machine. The Code Builder Prompt in Section 4 is ready for direct use.